Relational Database Systems –

COSC265



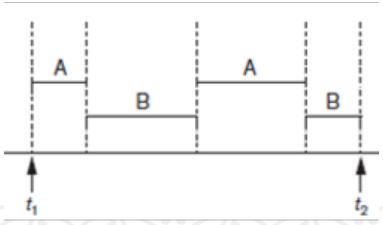
Lectures 29 & 30 – Transactions Processing

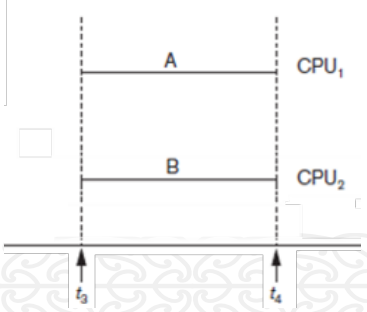
Miguel Morales

UC UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

Multiple Users

- **Single-user** systems
  - At most one user at a time can use the system
- **Multiuser** systems
  - Many users can access the system concurrently
- **Concurrency**
  - **Interleaved** processing: Concurrent **execution** of processes is **interleaved on a single** CPU
  - **Parallel** processing: Processes are **concurrently executed on multiple CPUs**





In-Confidence

Transactions

- A transaction is a **logical unit of database processing**
  - Could be **one or more statements**
  - An application program may contain several transactions

- The effects of transactions only become permanent when the transaction is **completed successfully**

- Functions of multi-user DBMSs
  - Transaction support
  - Concurrency control
  - Recovery Slide

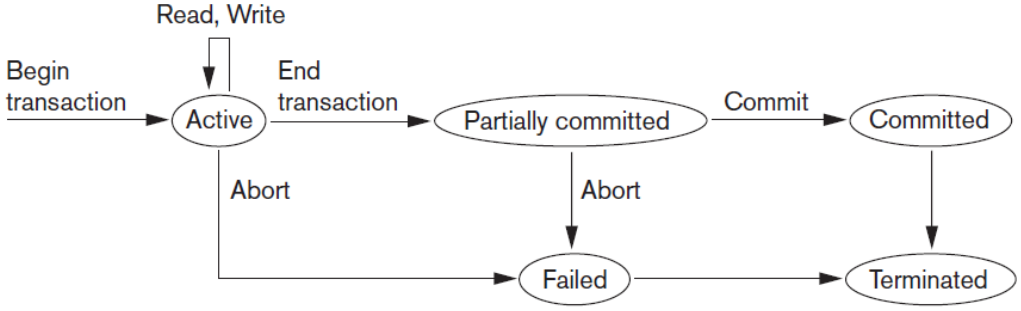


In-Confidence

Transaction Definition

- A transaction is an **atomic unit of work** that is either **completed in its entirety or not done at all**
  - For recovery purposes, the DBMS needs to keep track of when the transaction starts, terminates, and commits or aborts

- Transaction states:
  - **Active**
  - **Partially committed**
  - **Committed**
  - **Failed**
  - **Terminated**



In-Confidence

Transaction Basic Operations

- **read**_item(X)
  - Reads a database item named X into a program variable named X
  - The process includes finding the address of the disk block and copying to and from a memory buffer

- **write**_item(X)
  - Writes the value of program variable X into the database item named X
  - The process includes **finding the address** of the disk block, **copying to** and from a **memory** buffer, and **storing** the **updated** disk **block** back **to disk**

- Transactions executed by various users may **access and update the same database items**; this may cause ***problems***

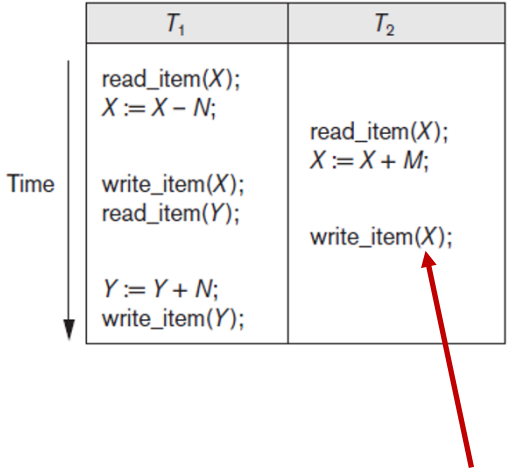
T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

In-Confidence

Concurrency Control

- The **Lost Update Problem**
  - Occurs when two transactions that access the same database items have operations interleaved
  - Results in an incorrect value for some database items



1. T1 is a Cowboys fan buying a ticket for the Monday Night Football game
2. Seat X is available, so they select it
3. T2 is an Eagles fan who also saw seat X
4. X is still *available,* so they choose it
5. T1, updates seat X as ***occupied by a Cowboys fan***
6. T1 continues with the allocation of the second ticket for the Cowboys fan. They click seat Y, which is next to seat X.
7. Meanwhile, T2 updates seat X as ***occupied by an Eagles fan.*** **T2 COMMITS**
8. Seat Y is available, so it is chosen
9. T1 updates seat Y as ***occupied by a Cowboys fan.*** **T1 COMMITS**

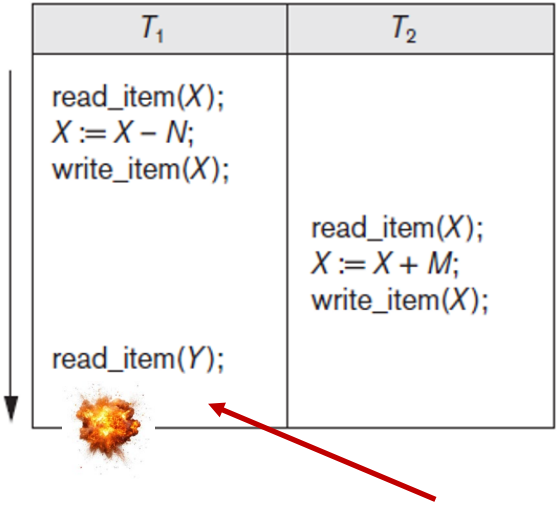
RESULT: Both transactions finished "successfully" (both committed), but now I have a Cowboys fan seated with an Eagles fan, and another Cowboys fan without a seat

Item X has an incorrect value because the update done by T_1 was lost (overwritten by T2)

In-Confidence

Concurrency Control

- The Temporary Update (or **Dirty Read**) Problem
  - This occurs when **one transaction updates** a database item **and then it fails** for some reason
  - **The updated item is accessed by another transaction** before it is changed back to its original value



1. T1 is an airline employee applying promotional prices (discounts) to some routes, CHC-SIN in this case.
2. The CHC-SIN flight price is reduced from $1200 to $900
3. T1 updates the flight price to $900
4. T2 is a traveller who just found out about the discounted price for CHC-SIN
5. The flight is added, and the total price of their reservation is updated
6. T2 **completes** the $900-flight purchase. **T2 COMMITS**
7. T1 continues applying discounts to other flights but discovers that those prices only apply if you are flying from AKL (not the CHC ones) and **aborts** the promotional updates. **T2 FAILS**, then **ROLLBACK**

RESULT: One transaction failed; the other succeeded. **T2 read uncommitted data from T1.** The $900 price was visible temporarily, but T1 rolled it back, so the price that T2 saw never actually existed in the committed database state.

T1 failed and the value of X must be changed back to its initial value.
Meanwhile, T2 has read a temporarily incorrect value of X.

In-Confidence

Concurrency Control

- The **Incorrect Summary** Problem
  - If **one transaction is calculating an aggregate summary** function on a number of records while **other transaction(s) are updating some of these records**
  - The aggregate function may calculate some values before they are updated and others after they are updated

- **Phantom Read** Problem
  - This occurs when a transaction **executes a query** returning a set of rows **twice**, **but the second execution returns a different set** of rows because another concurrent transaction inserted or deleted rows that match the query's search criteria

In-Confidence

Recovery Manager Operations

- **BEGIN**
- **READ**(X) or **WRITE**(X)
- **COMMIT**
  - This signals **a successful end** of the transaction so that any **changes** executed by the transaction can be safely committed to the database and **made permanent**
- **ROLLBACK**
  - This signals that the transaction has **ended abnormally**, so that any **changes** or effects that the transaction may have applied to the database **must be undone**
- **END**

In-Confidence

System Log

- The **system log** keeps track of transaction operations
  - Keeps a record of all transaction operations
  - Its data **permits recovery** from transaction failures
  - It is a **sequential**, **append-only** file
  - It is not affected by failure (except disk or catastrophic failure)
  - The log file is backed up periodically
  - The **last part** of the log file is **in main memory**

```
<T1, start>
<T1, write(A, old_value=10, new_value=20)>
<T1, write(B, old_value=30, new_value=40)>
<T1, commit>
...
```

- Recovery techniques use the following operators:
  - **UNDO:** Similar to rollback except that it applies to a single operation rather than to a whole transaction
  - **REDO:** This specifies that certain transaction operations must be redone to ensure that all the operations of a committed transaction have been applied successfully to the database.

In-Confidence

Checkpoint Record

- **Checkpoint:** Written to the log when the system writes to disk the effects of all write operations of committed transactions
  - This is done periodically (every *m* minutes or after *n* committed transactions)

- The steps:
  1. **Suspend** execution of all transactions
  2. **Force-write** all main memory buffers that have been modified to disk
  3. **Write a checkpoint** record to the log, and force-write the log to disk
  4. **Resume** executing transactions

In-Confidence

ACID Properties

- **Atomicity** (all or nothing)
  - A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all
- **Consistency**
  - A correct execution of the transaction must take the database **from one consistent state to another**
- **Isolation** (independence)
  - A transaction should **not** make its updates **visible** to other transactions **until** it is **committed**; this property, when enforced strictly, solves the temporary update problem
- **Durability** (persistence)
  - Once a transaction changes the database and the changes are **committed**, these **changes must never be lost** because of subsequent failure

In-Confidence

Transaction Schedules

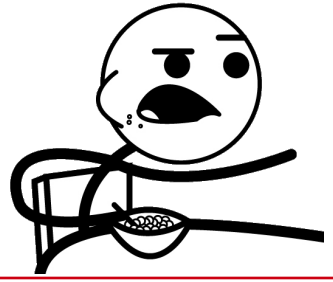
- Transaction **schedule**
  - It is **the order *S* of execution of operations** from various transactions

- **Serial** schedule
  - **All operations** of each transaction are executed **consecutively** in the schedule *S*
  - Every serial schedule is correct

- **Non-serial** schedule
  - An ordering of the operations of the transactions such that the operations of T_1 in *S* must appear in the same order in which they occur in T_1
  - Operations from other transactions can be interleaved with the operations of T_1

In-Confidence

Serial or non-serial?



UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

T_1	T_2

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);
	read_item(X);
	$X := X + M$;
	write_item(X);

Schedule A

T_1	T_2

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);
	read_item(X);
	$X := X + M$;
	write_item(X);

Schedule B

T_1	T_2

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);
	read_item(X);
	$X := X + M$;
	write_item(X);

Schedule C

T_1	T_2

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);
	read_item(X);
	$X := X + M$;
	write_item(X);

Schedule D

Conflicting Operations

- Two **conflicting operations** in a schedule are:
  - Operations that **belong to different transactions**
  - Operations that **access the same item** X
  - **At least one** of the operations **is a write_item(X)**

Conflicting operations can cause problems.

Can we make all of the transactions serial?
Can we *isolate* them?

- **Isolation levels:**

	Type of Violation		
Isolation Level	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

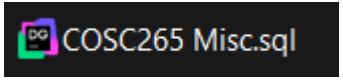
In-Confidence

Locks

- A **lock** is a **variable** associated with a data item describing the status for operations that can be applied
  - There is one lock for each item in the database

- **Binary locks**
  - Locked (**1**) – the item cannot be accessed
  - Unlocked (**0**) – the item can be accessed when requested

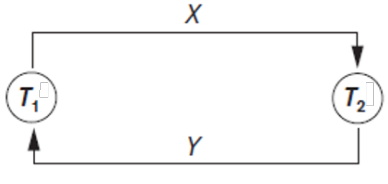
- Lock manager subsystem
  - Keeps track of and controls access to locks
  - Rules enforced by the lock manager module
  - SELECT * FROM **V$LOCK**;



Problems with Locks

- **Deadlocks**
  - Two or more transactions wait for each other

    T_1 waits for T_2, while T_2 waits for T_1
  - Both transactions are **stuck in a waiting queue** (circular waiting)
  - In these cases, **a victim needs to be chosen and aborted it**



- **Starvation**
  - A transaction keeps waiting because others keep getting priority

    T_3 waits while T_1, T_2, T_4, T_5 repeatedly get the resource T_3 needs
  - Caused by **unfair scheduling/resource allocation**
  - Use **fair scheduling or aging**

In-Confidence

Deadlock Prevention

- Every transaction **locks all items it needs in advance**

Almost unfeasible and very inefficient

- **Timestamps** (*If transaction T_i requests an item locked by T_j …*)
  - **Wait-die** – Older transactions wait; younger transactions die (are aborted)

Requesting transaction	Lock holder	Action
Older	Younger	Wait
Younger	Older	Die (rollback)

  - **Wound-wait** – Old wounds, young waits

Requesting transaction	Lock holder	Action
Older	Younger	Wound (abort) younger
Younger	Older	Wait

  - Concurrency control techniques based on timestamps do not use locks, **so deadlocks cannot occur**

In-Confidence

Starvation Prevention

- **First-come-first-served** queue
- **No waiting** algorithm
  - If a transaction is unable to obtain a lock, it is aborted and restarted later
- **Timeouts**
  - If the wait is longer than a predefined time, the transaction is aborted

Privileges

- **GRANT**
  - Gives a user permission to perform an operation
  - Giving mem168 permission to read the aircraft table:

```
GRANT SELECT ON aircraft TO mem168;
```

  - Give mem168 permission to modify the aircraft table

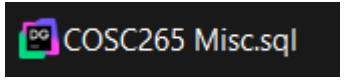
```
GRANT UPDATE ON aircraft TO mem168;
```

- **REVOKE**
  - Removes a permission from a user
  - Removing the UPDATE permission

```
REVOKE UPDATE ON aircraft FROM mem168;
```

  - Removing the SELECT permission

```
REVOKE SELECT ON aircraft FROM mem168;
```



In-Confidence

Privileges

- **GRANT WITH GRANT OPTION**
  - It allows a user to give the same privilege to another user
  - Giving *mem168* SELECT permission and allow *mem168* to grant it to others

```
GRANT SELECT
ON aircraft
TO mem168
WITH GRANT OPTION;
```

  - Now *mem168* can grant SELECT to another user
  - **Cascading revocation:** If the original privilege is revoked from *mem168*, Oracle also revokes privileges that *mem168* granted to other users

In-Confidence